UNIT-I: Introduction to JAVA

Introduction:

- JAVA is general purpose OOP (Pure OOP, since supports almost all OOP concepts) language developed by "Sun Microsystems" of USA in1991
- 'James Gosling' was the inventor (Developer/Creator) of JAVA language.
- Originally or firstly JAVA was named as "Oak" (Oak is name of tree which was found in front of Goslings Office)
- Basically, JAVA was designed for the development of software's for electronic devices like TV's, washing machine, VCR's, set-top box, Toasters etc.
- Java runs on a variety of platforms, such as Windows, Mac OS, and the various versions of UNIX.
- The 'C' and 'C++' languages has limitations in terms of several OOPs concepts which are not fully supported like **persistence** (Normally, when a program ends, all objects in memory are lost. With persistence, an object's data can be: saved to a **file or** stored in a **database**), **interface**, **garbage collector** etc.
- C and C++ are not as much reliable and portable therefore the new language JAVA is introduced to overcome the drawbacks of 'C' and 'C++'. Thus, JAVA made really simple, reliable, portable and powerful language.

History OR Evolution of JAVA:

Following table shows some milestones happen in developing of JAVA language:

Year	Development
1990	Sun Microsystem decided to develop special software that could be used to manipulate with electronic devices. A team of Sun Microsystems programmer headed by James Gosling was formed to undertake this task.
1991	After exploring the possibility of such idea, the team announced a new programming language called 'Oak' (Oak was the first name for JAVA)
1992	In this year, team of Sun Microsystems actual implements their language in home appliances like Microwave Oven etc. with tiny touch-sensitive screen.
1993	In this year, team of Sun Microsystems came up with new idea to develop web-based application that could run on all types of computers connected to Internet. For that, they create 'applet' (tiny program run on Internet by the browser)
1994	In this year, team of Sun Microsystems developed web browser called "HotJava" to locate and run applet on Internet.
1995	"Oak" was renamed as "JAVA" due to some legal snags (problems). Also, many popular companies like Netscape and Microsoft announced to support for JAVA
1996	Sun Microsystem releases Java Development Kit 1.0 (JDK 1.0) to develop different kinds of software.
1997	Sun Microsystem releases Java Development Kit 1.1 (JDK 1.1)
1998	Sun Microsystem releases JAVA 2 with JDK 1.2 of Software Development Kit (SDK 1.2).
1999	Sun Microsystem releases standard Edition of Java which was called J2SE(Java 2 Standard Edition) and J2EE (Java 2 Enterprise Edition)
2000	J2SE with SDK 1.3 was released
2002	J2SE with SDK 1.4 was released
2004	J2SE with JDK 5 (JDK 1.5) was released

- The latest version of Java is JDK 25, which was released on September 16, 2025
- Java is now under administration of Oracle organization.

Features or Characteristics or Advantages of Java:

- **Compiled & Interpreted:**
  - Usually, programming language is either compiled or interpreted. But Java combines both approaches that make Java 'two-stage system'.
  - In case of Java, First Java compiler translates or converts Java source program into 'byte code' (Byte code is **not** machine instruction code. Byte code is a low-level (e.g. Load a, Add a etc.), platform-independent instruction set generated by Java designed to run on a virtual machine (JVM)) And byte code file having extension '.class'
  - After compilation, Java Interpreter executes this byte code & we will get our desired output.

    Thus, we can say that Java is Compiled & Interpreted language.
- **Object Oriented:**
  - Java is pure object oriented language that supports for all OOP's concepts.
  - Almost, In Java, everything is an Object. All data and methods are resided (exist in) within an object and classes.
  - The object model in Java is easy to extend because it supports for Inheritance concept.
- **Portable and Platform independent:**
  - Portable: We know that, after compilation of Java source program it produce ".class" file i.e. byte code which is not machine dependent that's why such file is easily moved or transferred from one computer to another computer and hence Java is Portable.
  - Platform independent: After generation of byte code (.class file), this byte code is easily interpreted or executed on different kinds of computers having different platforms (Computers having different Operating system like windows, Linux, Mac OS etc. and different processors etc.)
- **Simple:**
  - Java is designed in such a way that it would be easy to learn since, most of syntax of java is same as C and C++.
  - If you understand the basic concepts of OOP then it is easy to implement in Java language.
- **Secure:**
  - We know that, most of viruses are attacked on files having extension '.exe', '.doc', '.gif', '.mpg' etc. but after compilation of Java source program it produce ".class" file i.e. byte code and which is virus free. And hence, Java enables us to develop virus-free, tamper -free applications.
- **Architectural-neutral:**
  - Java compiler generates an architecture-neutral class file format which makes the compiled code to be executable on **many processors**, with the presence of Java runtime system.
- **Robust:**
  - Java is strict type checking language which checks an error at both times i.e. at compile time and also at run time of program.
  - Due to this ability of checking errors at run time (exception Handling), we can eliminates any risk of crashing the system using Java and hence it is robust.
- **Multithreaded:**
  - Multithreaded means handling multiple tasks (jobs) simultaneously (at one time).
  - Java supports for multithreaded programs that means we need not wait for the application to finish its task before beginning another.
  - That is using Java, we can run multiple java applications without waiting to finish another.
- **Distributed:**
  - Java enables us to make such applications that can open and access remotely over the internet or network.
  - That is, multiple programmers at multiple remote locations are capable to work together on single project. That's why Java is distributed.
- **Dynamic and Extensible:**
  - Dynamic: Java is dynamic language which is capable to link new class libraries, methods and objects dynamically.

- Extensible: Java supports to write functions in C or C++ language such functions are called "native methods" and then we can add or link these methods with Java such that they can be used in many applications.

- **Ability to Deal with Database:**
- Java supports for JDBC (Java Database Connectivity) to send & retrieve data in tabular format with the database thus with the help of Java we are able to deal with database.

- **Automatic Memory Management:**
- We know that 'memory' is very important thing while dealing with the computer and we have to manage it very efficient manner.
- Java language supports for 'Garbage Collector' that automatically manages all the memory in efficient manner.

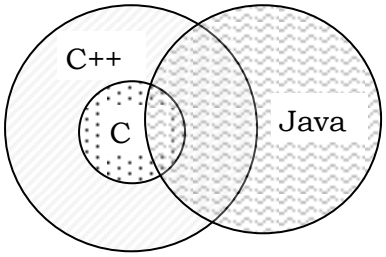
Limitations or Disadvantages of Java:

- **Slow language:**
  As compared to C and C++ languages, Java language compiler took much more time to compile the program & also Java interpreter took much more time to interpret the program that's why Java is slow language.
- **Strict type checking language:**
  Due to strict type checking, Java language checks much run time errors & that's why Java application took much time to execute.
- **Case sensitive language:**
  Due to case sensitive language, we must have to use correct spelling of inbuilt methods, classes, interfaces, keywords etc. while doing programming.
- **Not support for Multiple Inheritance:**
  Java does not support for Multiple Inheritance **but we can implement it by using 'interface'**.

Difference Between C++ and Java:

'C++' Language	Java Language
1) It is not pure OOP language.	1) It is pure OOP language.
2) It supports for template classes.	2) It does not support for template classes.
3) It supports for 'Multiple inheritance'	3) It does not supports for Multiple Inheritance but we implement it using 'interface'
4) It supports for global variable.	4) It does not have global variable
5) It supports for 'pointer'	5) It does not supports for 'pointer'
6) It supports for "destructor"	6) It does not supports for "destructor" but it is replaced by finalize( ) method.
7) It has 'goto' statement.	7) It has not 'goto' statement.
8) It has preprocessor directive statements like #define, #include etc.	8) It has no preprocessor directive statements like #define, #include etc.
9) It has three access specifiers viz: public, private and protected	9) It has four access specifiers viz: public, private, protected and default.
10) It supports for operator overloading.	10) It does not supports for operator overloading
11) Automatic memory management is not supported.	11) Automatic memory management is supported by 'Garbage Collector'.

Following fig. shows overlapping of C, C++ and Java:



From above fig.

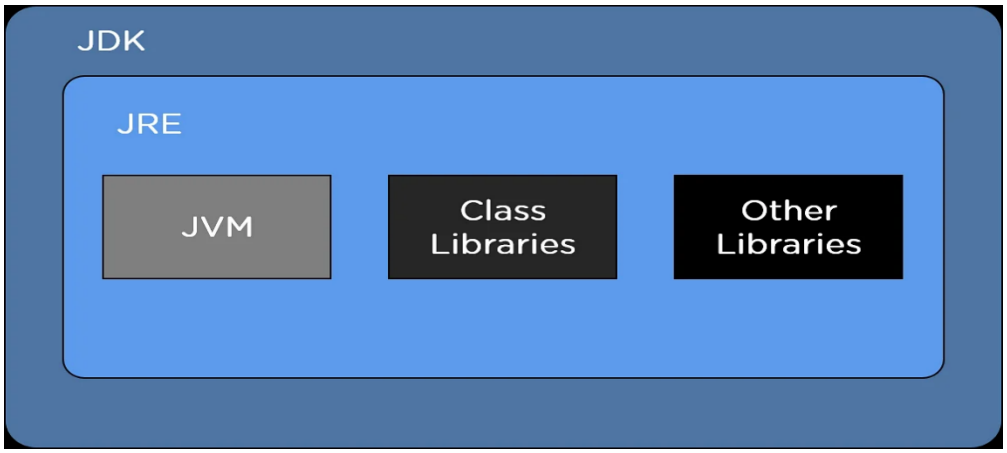
- We know that, 'C++' is superset of 'C' language therefore every 'C' program is easily executed by 'C++' compiler.
- But, Java language is partly combination of 'C' and 'C++' language and it having its own extra features therefore Java can be considered as first cousin of 'C++' and second cousin of 'C'

Java Development Kit (JDK):

- JDK in Java is an essential component necessary to develop JAVA programs or JAVA software's.
- It is technically an implementation of either Java Standard Edition (J2SE) or Java Enterprise Edition (J2EE).
- JDK is a bundle of **software development tools and supporting libraries** combined with the Java Runtime Environment (JRE) and Java Virtual Machine (JVM).
- JSL (Java Standard Library) also called as Java API (Application Programming Interface) is the main part of JDK that contains thousands of **Packages** (group related classes, interfaces, enumerations, methods and sub-packages etc. in a structured hierarchy).

The Architecture of JDK in Java:

- The architecture of JDK in Java includes the following modules as described in the image below.

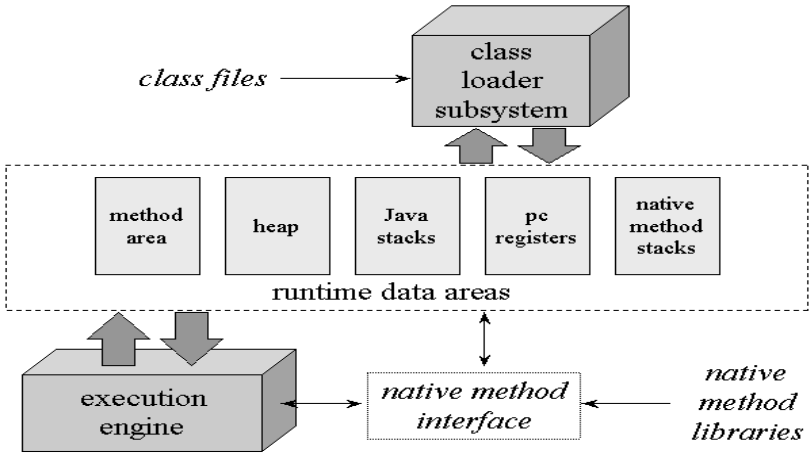


- The two vital software modules of JDK are:

1) JVM (Java Virtual Machine):

- Java Virtual Machine is a software tool responsible for **creating a run-time environment** for the Java source code to run. The very powerful feature of Java, "Write once and run anywhere," is made possible by JVM.
- The JVM stays right on top of the host operating system and process the Byte Code (machine language), such that it would easily execute by microprocessor.
- Java Virtual Machine plays vital or important role in execution of java program therefore it is *heart* of java.

Following Figure shows **Architecture of JVM**:



Note:

- *Java Compiler generates or creates byte code which is machine independent or platform independent therefore it is easily interpreted by any JVM that's why it is called as "write once run anywhere".*
- *But, JVM is platform dependent i.e. windows, Linux, Mac OS, Unix etc. operating system having their different-different JVM's.*

Working of JVM:

- First of all, java source file (.java file) is converted into byte code (.class file) by the java compiler and this byte code file is given to the JVM.
- In JVM, there is one module or program called 'Class loader sub system' which performs following functions:
  - First, 'Class loader sub system' loads the '.class' file into memory.
  - Then it verifies whether all byte code instructions are correct or not.
  - If it finds some problem in byte code then it immediately terminates the execution.
  - If byte code is proper or correct then it allocates necessary memory to execute the program.

Also, this memory is divided into 5 parts called 'Runtime data area' & these parts as follows:

1) **Method area:**
   In this memory area, all class code, variable's codes, method's codes etc. are stored.
2) **Heap:**
   In this memory area, all objects are created and stored.
3) **Java Stacks:**
   Actually, java methods are stored in 'Method area' but actual execution of such java methods are happened under 'Java stacks' area.
4) **PC registers:**
   This area contains the memory addresses of instructions of the methods.
5) **Native method stacks:**
   All native methods (C, C++ functions) are executed under native methods stacks.
   And all native methods are connected with JVM by '*native method interfaces'*

After, allocation of memory into corresponding parts then it comes towards 'Execution Engine'.

- Execution Engine can consists of two things VIZ:
  1) Interpreter 2) JIT (Just In Time) compiler.

- This **interpreter and JIT compiler** are responsible for converting byte code into **machine instruction** such that it easily executed by microprocessor (CPU).
- After, loading the ".class" file into memory, JVM first identifies which code is to be left to interpreter and which one to JIT compiler so that the performance will be better.
- The blocks of code allocated for JIT compiler are also called 'hotspots'. Thus, the interpreter and JIT compiler will work simultaneously to translate the byte code into machine instructions.

Note that: JIT compiler is a part of JVM which increases execution speed of program.

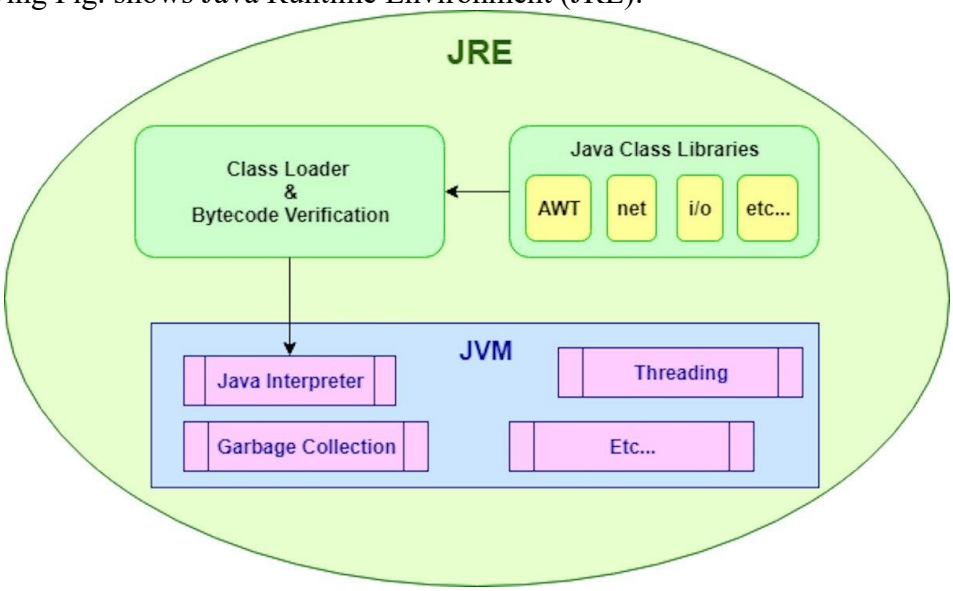
2) JRE (Java Run-time Environment)

- Java Run-time Environment is a software platform where all the Java Source codes are executed.
- JRE is responsible for integrating the software plugins, jar files, and support libraries necessary to run the java source code.
- The Java Runtime Environment, or JRE, is a software layer that runs on top of a computer's operating system software and provides the class libraries and other resources to run java source code.
- A Java™ runtime environment (JRE) is a set of components to create and run a Java application.
- A JRE is part of a Java development kit (JDK).
- A JRE is made up of a Java virtual machine (JVM), Java class libraries, and the Java class loader.
- In short JDKs are used to develop Java software whereas JREs provide programming tools and deployment technologies.

Why use a Java runtime environment?

- We know that to execute a program, there is need of an environment and JRE provides such environment because JRE usually resides on an operating system (OS) like Linux, Unix, Microsoft Windows, or MacOS.
- Because of JRE, java programs are controlled the capabilities of the OS and its resources (such as memory and program files).
- A JRE acts as facilitator or interface between the Java program and the OS that demands resources towards the OS.

Following Fig. shows Java Runtime Environment (JRE):



JDK Components (JDK tools):

Following is the list of tools or components of JDK which are used to develop and run the java programs:

Sr.NO	Tool or Component	Description or Use
1	javac (Java Compiler)	It translates or converts java source program into byte code file & that file understood by java interpreter
2	java (java Interpreter)	It runs java applications by reading corresponding byte code file & gives result.
3	Appletviewer	It runs or views java applets onto the web browser.
4	javap (Java disassembler)	It converts byte code file into program description.
5	Javadoc	It creates or produces HTML format documentation of java source file. But it needs public class for documentation.
6	Javah	It creates or produces header files for use of native methods.
7	jdb (Java Debugger)	It helps us to checks errors in java program.

Why Java does not support for pointer?

→

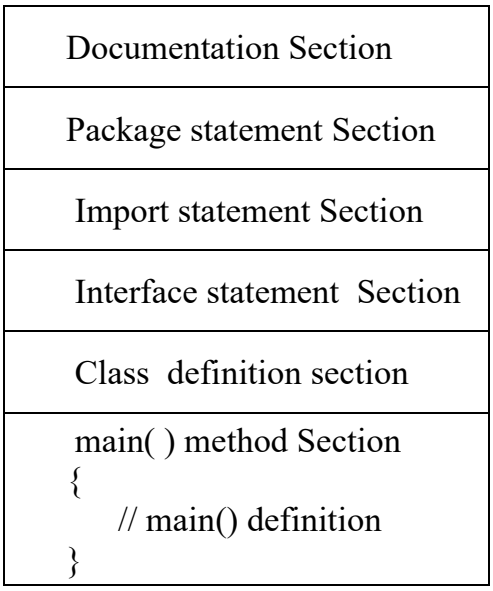
1) We know that 'pointers' are used to hold memory address. And most of viruses are trying to attack on memory that's why Java does not support for pointer and hence Java is secured.

2) Also, pointers are helpful for dynamic memory allocation i.e. it is used for run time memory management, but in Java all memory management is automatically done by '**Garbage collector'** that's why no need of pointer.

Structure of JAVA Program:

- We know that single Java program may contains multiple classes but out of these classes, one class should be public and that class contains main( ) method from which JVM interprets the byte code.
- Note that: Java is pure OOP language i.e. all programs must have classes and objects.

- A typical Java Program is divided into several Sections which are shown in following figure:



1) Documentation Section:

- This section contains set of comments lines showing details of java source program such as program name, programmer name, date of program, version etc. this help program readability.

In Java, we can give comments by three ways VIZ:-

1) Single line comment:

- If we have to specify general information of program within single line then single line comment is used. Single line comment is given by // notation.
- Also, we can specify this comment anywhere in program.

E.g. // Program Name= Addition of two numbers.

2) Multiline comment:

- If we have to specify general information of program within multiple lines then multi line comment is used. Multiline comment is given by following notation.

```
/* ----------
   ----------
---------- */
```

E.g.

```
/*      Program Name:   Multiplication
        Programmer: James Gosling */
```

3) Third Style comment: (Java documentation Comment)

- This type of comment is specially used for documentation purpose.
- If we specify description using 'Third style comment' then it is shown in HTML files created by using 'Javadoc'
- This comment is used to provide description for every feature in Java source program.

Third Style comment is given by-

```
/**  ----------------------
     ----------------------   */
```

E.g.

```
/** This class is used for addition  */
public    class      add
{
      /** This method is used for addition  */
        public   void  addition( )
        {
              // statements
        }
}
```

- In above example, two times documentation comment is used that will show description of class 'add' and description of method 'addition( )' in HTML file. Note that: For generation of HTML documentation of java source program using 'javadoc' component, class and method should be public or protected.

2) Package statement Section:

- This section is used to declare our own package. When we declare own package then it informs to the java compiler to link all classes of our package with java source program.
- Syntax to specify package statement:

```
package     package_name;
```

E.g.

```
package     student;
```

More about package will be discussed in next chapter.

3) import statement Section:

- In this section, we can import existing package in our java source program.
- We know that, in case of 'C' language if we have to use printf( ) method then we include 'stdio.h' header file using preprocessor directive '#include'.
- Similarly, if we have to use existing classes or exiting methods of JSL (Java Standard Library) then we have to import that package in our source program using *'import'* statement.
- Syntax to import package in program:

```
import      package_name;
```

E.g.

```
import      java.lang.* ;        //It is default package so need to import
```

Difference between #include & import:

→

- *When we include header file in program then C/C++ compiler goes to the standard library (it is available at c:\tc\lib) and searches for included header file there. When it finds the header file, it copies entire header file into the program where the #include statement is written. Thus, if our C/C++ program has only 10 lines still C/C++ compiler shows hundreds of line compiled this is due to copy of included header file at #include statement. Therefore, our program size increases & hence* ***it causes memory wastage****.*
- *When we import package in Java program then, JVM checks whether imported package is present in JSL or not. If JVM finds imported package then it executes corresponding method code there and only returns its result to source program therefore size of source program in not increased as happened in C/C++. And hence,* ***memory wastage is solved****.*

4) interface statement Section:

- In this section we can define interfaces.
- Interfaces are similar to the classes but all methods of interface are by default 'public' and 'abstract'
- This is optional section, used while implementing multiple inheritance in java.

5) class definition Section:

- We know that, single Java program may contain multiple classes and every class has its own attributes (data members) and methods (member functions). Such type of classes can be defined under class definition section.

  Note that:

  - *Java program may contain multiple classes but out of these classes, only one class should be public and that class have main( ) method from which JVM interprets the byte code.*

6) main( ) method Section:

- We know that in case of C/C++, main( ) function is compulsory from which execution of program starts. Like that java program also have main( ) method from which JVM starts program interpretation. This is compulsory section. Also, **main( )** method in Java must be **public**.

- If we made **main( )** as **private or protected or default** then it is **not assessable for JVM** also.
- main( ) method should be defined under any class of program but that class should be public.

Simple Java Program:

Let's consider following simple java program;

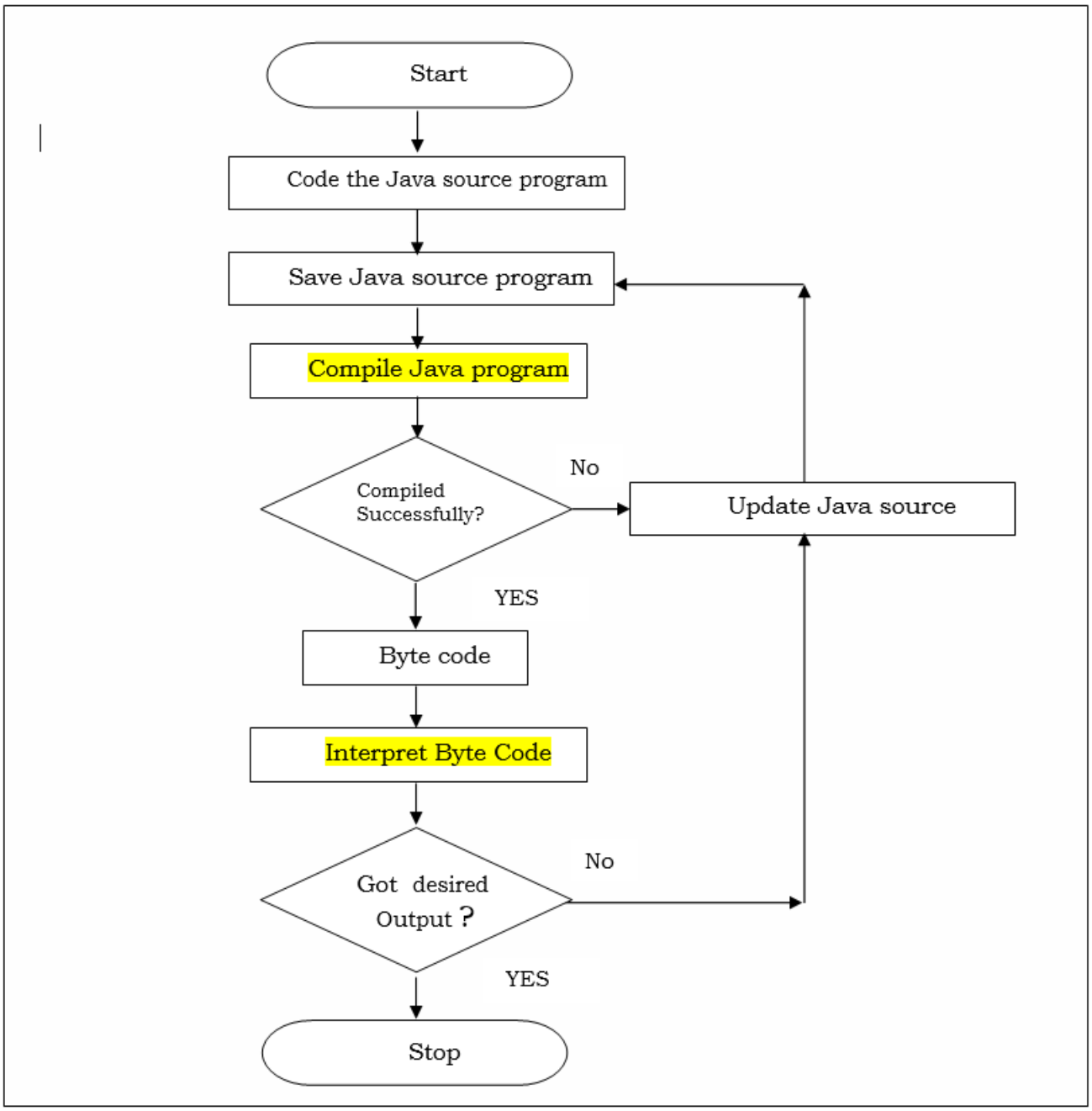
```
import     java.lang.*;
class                first
{
    public   static  void   main(String   args[ ])
    {
            System.out.println("Welcome in JAVA programming");
    }
}
```

In above program,

- "**java.lang**" is a **package** which is imported using 'import' keyword. This package contains lots of inbuilt classes such as System, String, Integer, Float etc. This is **default package i.e. there is no necessary to import it.**
- Here, main( ) method is compulsory which is declared as public, static and void
  - It is **public** because it made available for JVM for interpretation of java program.
  - It is **static** because it should be called without any object; it is invoked by JVM with class name
  - It is **void** because it does not return any value.
- Also, main( ) method accepts array of string as argument or input which is called as '**command line argument'** The passed values are stored in args[ ] array at **individual indices starting from index 0**
- System.out.println( ) statement:
  - "System" is inbuilt wrapper class which was found under 'java.lang' package.
  - "out" is an object of 'System' class which is 'static' & hence it is accessed by 'System' class name.
  - "println( )" is a method was found in "System" class used to display output and called by using "out" object.

Steps to execute Java Program:

Following flowchart shows compiling and interpreting Java program;



Checking JAVA (JDK) installation:

While doing java programming, first check JDK i.e. JAVA is installed on your system or not.
It is checked by running **java -version** command on CMD prompt which is shown as follow-

```
C:\Users\Admin>java -version
java version "1.8.0_131"
Java(TM) SE Runtime Environment (build 1.8.0_131-b11)
Java HotSpot(TM) Client VM (build 25.131-b11, mixed mode)
```

Syntax to Compile Java Source program:

Java program is compiled with 'java source program' name along with 'javac' component which is given as fallow:

```
javac    JavaSourcePgmName.java
```

E.g. Consider, we have *'good.java'* source program then we can compile it as follow:

```
javac        good.java
```

If *'good.java'* program have one class named 'good' then *'good. Class'* byte code is generated.

Syntax to interpret or Run or Execute the Byte code:

Java program is interpreted or run or execute using byte code (.class file) along with 'java' interpreter which is given as fallow:

```
java        ByteodeFile
```

E.g. Consider, we have *'good.class'* byte code then it is interpreted as follow:

```
java        good
```

Note that: *After compilation of java source program, byte code (.class file) is generated. And then JVM interpret that byte code and we got our result.*

Syntax to pass arguments to main( ) method while interpretation of bytecode :

We can also pass some string type arguments to main ( ) method called 'command line arguments' using following syntax.

```
java        ByteCodeFile        arg1        arg2  - - - -    argN
```

In above syntax;
arg1, arg2, - - - - ,argN are the command line arguments passed to main() method while interpreting.
Note that: All passed arguments are stored in formal parameter (String type array) of main( ) method at individual indices.
E.g. Consider following Program:

```
import     java.lang.*;
class                first
{
    public   static void   main(String   args[ ])
    {

            System.out.println("FirstName= "+ args[0]);
            System.out.println("MiddleName= "+ args[1]);
            System.out.println("LastName= "+ args[2]);


    }
}
OUTPUT:
            javac      first.java

            java      first          SACHIN     RAMESH     SHINDE
```

In above example; three command line arguments are passed to main( ) method. They are SACHIN RAMESH SHINDE.
All these arguments are stored in 'args' String type array in main( ) method at individual indices as fallow;

args[0]=>	SACHIN
args[1]=>	RAMESH
args[2]=>	SHINDE

Also, we use '+' operator to concatenates two strings with each other.

Naming Conventions Used in Java:

- *Naming Conventions* specifies the rules to be followed by java programmer while writing or coding java source program.
- We know that java program contains the package, classes, interfaces, methods, variables etc. and all these have separate naming conventions they are as follow:

Naming Conventions for Package:

- We know that, Package is one kind of directory that contains the classes and interfaces etc.
- Package name in java should write in **small letters only**.

Example:

java.lang
java.awt
javax.swing

Naming Conventions for class or interface:

- Class and interface name in java **should start with capital letter**.

Example:

System
String
Integer
Float etc.

Naming Conventions for methods:

- The first word of a method name is in small letters, then from second word onwards, each new word starts with capital letter as:

  Example:

  println();
  read**L**ine();
  get**N**umber**I**nstance();

Naming Conventions for variables:

- Naming conventions for variable is same as that of methods i.e. *The first word of a variable name is in small letters, then from second word onwards, each new word starts with capital letter as:*

  Example: age
  emp**B**ame
  emp**N**et**S**al

Java Tokens:

- "Token is nothing but smallest individual unit of java source program."
- We know that Java is pure OOP language i.e. every program has classes and every classes has some methods and methods contains executable statement and every executable statement contains the tokens i.e. statements are made up of several tokens.
- Following are the several tokens in Java program:

  1) Keywords 2) Data type 3) Identifier 4) Variable
  5) Constant or Literals 6) Operators 7) Special symbols.

Let us see all tokens in details:

1) Keywords (Reserve words):

- The words whose meaning is already known by java compiler are called as 'Keywords'.
- These words having fix meaning and we are not able to change that meaning therefore they are also called as 'Reserve Word'.
- Java language contains more than 50 keywords and they are listed as fallow:

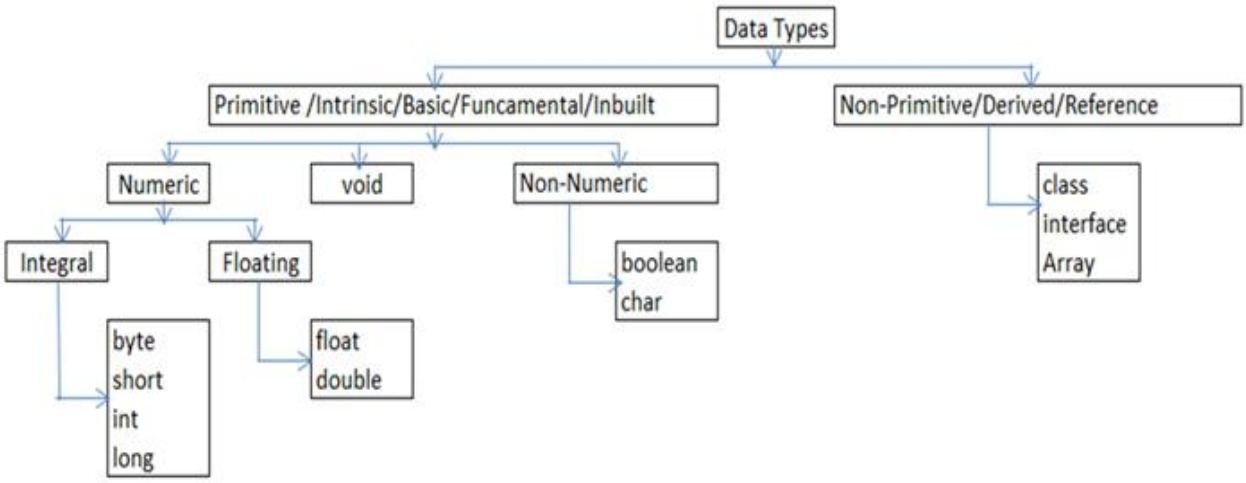
abstract	continue	for	new	switch
assert	default	if	package	synchronized
boolean	do	implements	private	this
break	double	import	protected	throw

byte	else	instanceof	public	throws
case	enum	int	return	transient
catch	extends	interface	short	try
char	final	long	static	void
class	finally	native	strictfp	volatile
float	super	while		

2) Data Type:

- Data: "Data is nothing but collection of raw information or unprocessed information that we provide for the computer for processing"
  e.g. numbers, string, alphanumeric etc.
- Data Type:
- Concept: When we give data to the computer for processing at that time compiler does not know which type of input data is.
- Generally, Data types are used to tell the compiler which type of input data is.
- *Definition*: "Type of Data is called as Data Type"

Following tree diagram shows data types in Java language:



Let us see all these data types in details:

Primitive Data Types:

There are nine primitive data types supported by Java. Primitive data types are predefined by the language and named by a keyword.

1) byte:

- byte data type is an 8-bits (1 byte) integral data type.
- Its Minimum range value is -128 (i.e. -2^7)
- Its Maximum range value is 127 (inclusive)(i.e. 2^7 -1)
- Its Default value is 0
- byte data type is used to save space in large arrays, mainly in place of integers, since a byte is four times smaller than an int.
- Example: byte a = 100; byte b = -50;

2) short:

- short data type is a 16-bits (2 bytes) integral.
- Its minimum range value is -32,768 (i.e. -2^15)
- Its Maximum value is 32,767 (inclusive) (i.e. 2^15 -1)
- Short data type can also be used to save memory as int data type. A short is 2 times smaller than an int
- Its Default value is 0.
- Example: short s = 10000, r = -20000;

3) int:

- int data type is a 32-bits (4 bytes) signed integral data type.
- Its Minimum range value is - 2,147,483,648.(i.e. -2^31)
- Its Maximum range value is 2,147,483,647(inclusive).(i.e. 2^31 -1)
- int is generally used as the default data type for integral values unless there is a concern about memory.
- Its default value is 0.
- Example: int a = 100000, b = -200000;

4) long:

- long data type is a 64-bits (8 bytes)signed integral data type.
- Its Minimum range value is -9,223,372,036,854,775,808.(i.e. -2^63)
- Its Maximum range value is 9,223,372,036,854,775,807 (inclusive). (i.e. 2^63 -1)
- This type is used when a wider range than *int* is needed.
- Its Default value is **0L**.
- Example: long a = 100000**L**, long b = -200000**L**;

5) float:

- float data type is a single-precision 32-bits (4 bytes) floating data type.
- Its Minimum range value is - $3.4e^{38}$ to $-1.4e^{-45}$ for negative value.
- Its Maximum range value is $3.4e^{38}$ to $1.4e^{-45}$ for positive value.
- Float is mainly used to save memory in large arrays of floating point numbers.
- Its default value is **0.0f.**
- Float data type is never used for precise values such as currency.
- Example: float f = 234.5**f**;

6) double:

- double data type is a double-precision 64-bits (8 bytes) floating data type.
- Its Minimum range value is $-1.8e^{308}$ to $-4.9e^{324}$ for negative value.
- Its Maximum range value is $1.8e^{308}$ to $4.9e^{324}$ for positive value.
- This data type is generally used as the default data type for decimal values, generally the default choice.
- Double data type should never be used for precise values such as currency.
- Its Default value is **0.0d**.
- Example: double d = 123.4;

7) boolean:

- boolean data type represents *one bit* of information.
- There are only two possible values: *true and false*.
- This data type is used for simple flags that track true/false conditions.
- Its default value is *false*.
- Example: boolean one = true;

8) char:

- char data type is a single 16-bits (2 bytes) non-numeric character data type.
- Its Minimum value is '\u0000' (or 0).
- Its Maximum value is '\uffff' (or 65,535 inclusive).
- Char data type is used to store any character.
- Example: char letter ='A';

9) void:

- void means no value
- This data type is generally used to specify return type of method.
- If return type of method is void then that method does not return any value.

Non-Primitive or Derived or Reference Data Types:

- The data types derived or created with the help of inbuilt data types is called 'Non-primitive or derived or reference data types"
- Java language has three Non-primitive data types viz. **array, class and interface**.
- *Note- In next units we will discuss above mentioned non-primitive data types.*

3) Identifier:

- "Identifier is the name given by the programmer for any variable, package, class, interface, array, object etc."
- There are several rules to declare or define the identifier:
    1) Identifier should **not** be keyword.
    2) Identifier should **not start** with digit.
    3) Identifier can be **combination of alphabets, digits** or **underscore or dollar sign($)**.
    4) Identifier should not contain **special symbol except underscore and dollar sign($)** .
    5) Identifier should **not contain any white space character** like horizontal tab, vertical tab, new line etc.
    6) Identifier should be meaningful.
    7) Identifier can be of any length.

4) Variable:

- "Variable is the name given to the memory location where the data is stored such quantity is called as Variable"

OR

- "The quantity that changes during program execution is called as Variable"
- Concept: The main concept behind variable is that every variable has an ability to store the data.

Syntax to declare variable:

```
DataType    variableName ;
```

Here;

DataType is any valid data type in 'Java' language.
variableName is an **identifier.**

Example:
```
int   rollno;
char  x;
```

- **There are several rules to declare the variable:**

1) Variable should not be keyword.
2) Variable should not start with digit.
3) Variable can be combination of alphabets, digits or underscore **or dollar sign($)**.
4) Variable should not contain special symbol **except underscore and dollar sign**.
5) Variable should not contain any white space character like horizontal tab, vertical tab, new line etc.
6) Variable should be meaningful.
7) Variable can be of any length.
8) Declared *local variable* must be initialized anywhere in block.

Types of Variables in java:

- **Local variables:**
    - The variables which are declared inside methods, constructors or blocks are called local variables.
    - These variables are declared and initialized within the method and they will be destroyed automatically when the method has completed its execution.
    - The scope of local variable is limited within block i.e. they are not accessed outside block.
- **Instance variables: (Non-static variable)**
    - Instance variables are non-static variables which are declared inside a class but outside any method.
    - These variables are instantiated when the class is loaded.

- Instance variables are accessed inside any non-static method, constructor or blocks of that particular class but **not** accessed within static method ***directly*.**
- **Object** is required to access instance variable inside static method.
- These variables are in the scope of object i.e. they are stored in object of class.

- **Class variables: (static variable)**
  - Class variables are variables which are declared within a class but outside any method with the **static** keyword.
  - These types of variables are **common to all objects** of class i.e. all static data are shared among all objects of class commonly. Hence, they are accessed with the help of class name.
  - **Note:** Such class variables are **not** in the scope of object i.e. they are not stored in object.

Following program shows variables in Java-

```
public    class    first
{
            int        a;            // instance variable
            static    float    b;   // class variable
    public   static void main(String    []arg)
    {
            boolean    flag=true;    //local variable
            first    obj=new    first();
            obj.a=75;
            first.b=45;
            System.out.println("Local variable="+flag);
            System.out.println("Instance variable="+obj.a);
            System.out.println("Class variable="+first.b);
     }
}
```

5) Constant (Literals):

- A *literal* represents a fixed value that is stored into variable directly in the program. They are represented directly in the code without any computation.
- Literals can be assigned to any primitive type variable.
  For example:
  - byte p = 68;
  - char a = 'A';

Java has different types of literals VIZ:
1) Integer Literals
2) String Literals
3) Character Literals
4) Float Literals
5) Boolean Literals

Let us see all literals in details:

1) Integer Literals:

- Integer literals represent the fixed integer values like 23, 78, 658, -745 etc.
- The data type byte, int, long, short belongs to decimal number system that uses 10 digits (from 0 to 9) or octal number system that uses 8 digits (from 0 to 7) or hexadecimal number system that uses 16 digits (from 0 to F) to represent any number.

Note that:
*Prefix **0 (zero) is used to indicate octal** and **prefix 0x (zerox) indicates hexadecimal** when using these number systems for literals.*
For example:

- int decimal = 100;
- int octal = **0**144;
- int hexa = **0x**64;

2) String Literals:

- String literals are collection of characters which are representing in between a pair of double quotes.

Example:

```
String   x="Hello World";
```

3) Character Literals:

- Character literals are characters which are representing in between a pair of single quotes.
- Character literals are like 'A' to 'Z', 'a' to 'z', '0' to '9'or Unicode character like '\u0042' or escape sequence like '\n', '\b' etc.

Example:

```
char   x= 'Z';
```

4) Float Literals:

- Float literals represent fractional values like 2.3, 86.58, 0.0, -74.5 etc.
- These types of literals are used with float and double data types.
- While writing such literals, we can use E or e for scientific notation, F or f for float literal and D or d for double literals (this is default and generally omitted)

Example:

```
float     p = 9.26;
double   q = 1.56e3;
float     m =986.8f;
```

5) Boolean Literals:

- Boolean literals represent only two values – true or false. It means we can store either 'true' or 'false' into a Boolean type variable.

Example:

```
boolean     p =true;
```

6) Operators:

An '*operator*' is a symbol that tells computer to perform specific task.

OR

An 'Operator' is a symbol that operates onto the operand to perform specific task.

Following are the several operators present in Java:

1) Arithmetic operators (+,-,*,/,%)
2) Relational operators (<,>,<=,>=,==,!=)
3) Logical operators (&&,||,!)
4) Increment and decrement operator (++,--)
5) Assignment operator (=)
6) conditional operator/ Ternary Operator (? :)
7) Bitwise operators
8) 'new' operator
9) 'instanceof' operator
10) cast operator

(Note that: All the operators listed above from 1 to 6 are same as C/C++ language therefore refer notes of C/C++ language)

Let's see some operators of Java language as follows:

7) Bitwise operators:

- Bitwise operators are used to manipulate data at bit (0 or 1) level.
- These operators act on individual bits of the operands.
- Bitwise operators only act on integral data types such as byte, int, short, long. That is, they are not worked on float and double data type.
- When these operators work on data then internally (automatically) data is converted into binary format & then they start their working.
- There are 7 different bitwise operators present in Java as follows:

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR (i.e. XOR)
~	Bitwise Complement
<<	Bitwise left Shift
>>	Bitwise Right Shift
>>>	Bitwise Zero fill Right shift

The truth table or working of bitwise operator **&,** | and ^ is shown in following table:

Op1	Op2	Op1 & Op2	Op1 | Op2	Op1 ^ Op2
1	**1**	1	1	0
1	0	0	1	1
0	1	0	1	1
0	0	0	0	0

Bitwise AND operator (&):

This operator performs 'AND' operation on individual bits of the numbers. To understand the working of *'&'* operator see following example.

E.g.:

1) 22&5

→

		128	64	32	16	8	4	2	1
22	→	0	0	0	1	0	1	1	0
5	→	0	0	0	0	0	1	0	1
22&5	→	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**0**

In above table, '1' bit is found in '4' column only therefore '22&5' gives result **'4'**

Bitwise OR operator (|):

This operator performs 'OR' operation on individual bits of the numbers. To understand the working of '|' operator see following example.

E.g.:

1) 35|7

→

		128	64	32	16	8	4	2	1
35	→	0	0	1	0	0	0	1	1
7	→	0	0	0	0	0	1	1	1
35|7	→	**0**	**0**	**1**	**0**	**0**	**1**	**1**	**1**

In above table, '1' bit is found in '1', '2', '4' and '32' columns therefore '35|7' gives result 1+2+4+32= **39**

Bitwise XOR operator (^):

This operator performs 'exclusive OR' operation on individual bits of the numbers. Its symbol is denoted by '^' which is called *cap, carat or circumflex* symbol. To understand the working of '^' operator see following example.

E.g.:1) 47^4

→

		128	64	32	16	8	4	2	1
47	→			1	0	1	**1**	1	1
4	→			0	0	0	**1**	0	0
47^4	→			**1**	**0**	**1**	**0**	**1**	**1**

In above table, '1' bit is found in '1', '2', '8' and '32' columns therefore '47^4' gives result 1+2+8+32=**43**

Bitwise complement operator (~):

This operator gives complement form of the given number. Its symbol is denoted by '~' which is called '*tiled*' symbol. To understand the working of '~' operator see following example.

E.g.: 1) ~47

→ It gives result= -48

2) ~(-26)

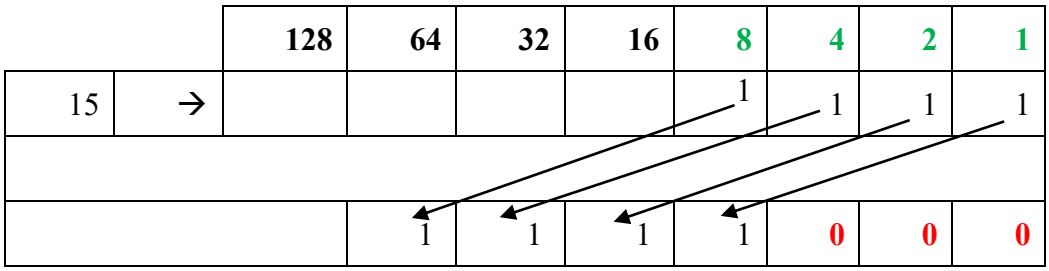
→ It gives result= 25

Bitwise left shift operator (<<):

This operator shifts the bits towards *left side* by a specified number of positions. Its symbol is denoted by '<<' which is called *double less than* symbol. To understand the working of '<<' operator see following example.

E.g.:1) 15<<3

→



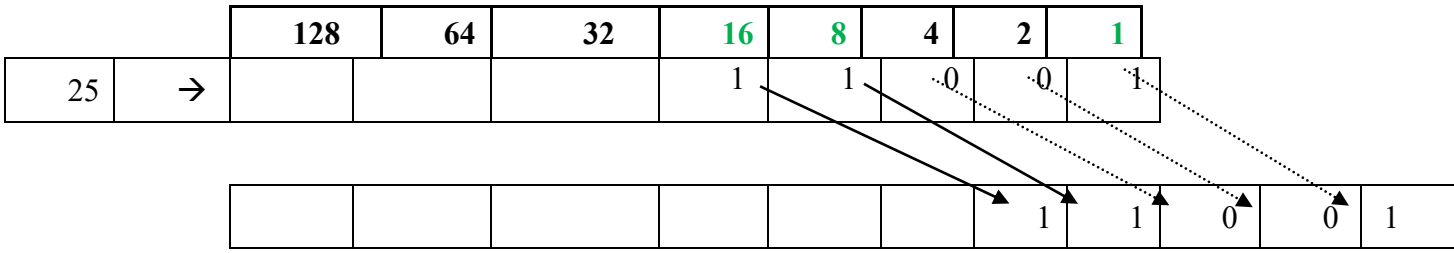
In above table, '1' bit is found in '8', '16', '32' and '64' columns therefore '15<<3' gives result 8+16+32+64=**120**

Bitwise Right shift operator (>>):

This operator shifts the bits towards *right side* by a specified number of positions. Its symbol is denoted by '>>' which is called *double greater than* symbol. To understand the working of '>>' operator see following example.

E.g.:1) 25>>3

→



In above table, '1' bit is found in '1', and '2' columns therefore '25>>3' gives result 1+2=**3**

Bitwise Zero Fill Right shift operator (>>>):

- This operator also shifts the bits towards *right side* by a specified number of positions. But, it stores '0' in the sign bit. Its symbol is denoted by '>>>' which is called trip*le greater than* symbol. Since, it always fills '0' in the sign bit therefore it is called *zero fill right shift operator*.
- In case of negative numbers, its output will be positive because sign bit is filled with '0'

8) 'new' operator:

- 'new' operator is used to *create object of class*.
- We know that, objects are created on 'heap' memory by JVM dynamically.

Syntax to create object:

```
className obj=new className( );
```

Here,

'className' is name of the class.

'obj' is name of created object which is an identifier.

Example: Consider, there is class named 'Employee' then we create its object as follow,

```
Employee    emp = new    Employee( );
```

Here, 'emp' is an object of class 'Employee'

9) 'instanceof' operator:

- 'instanceof' operator is used *to check created **object** belongs to particular **class** or not*.
- Also, this operator used *to check created **reference** belongs to particular **interface** or not*.

Syntax:

```
boolean    var = obj    instanceof    className;
                    OR
boolean    var=  ref    instanceof    interfaceName;
```

Here,

'var' is variable of boolean data type.
'obj' is object of class.
'ref' is reference of interface.
'className' is name of class.
'interfaceName' is name of interface.

Example:

```
boolean    x = emp    instanceof    Employee;
```

Here, 'instanceof' operator checks an object 'emp' is an object of class 'Employee' or not.
If 'emp' is an object is class 'Employee' then 'instanceof' operator return *'true'* otherwise it returns *'false'*

```
// Program that demonstrate use of 'instanceof' operator
class worker
{
}
class  sangola
{
    public static void main(String[] args)
    {
            worker  wk=new  worker();
            boolean  x;
            x=wk  instanceof  worker;
            if(x==true)
                    System.out.println("It is instance of Worker class");
            else
                    System.out.println("It is not instance of Worker class");
    }
}
OUTPUT:        It is instance of Worker class
```

10) 'cast' operator:

- cast operator is used to convert one data type into another data type.
- To convert data type of any variable or an expression, just we have to *specify conversion data type before variable or expression within simple bracket (braces)*.
- Syntax:

(datatype) Expression;

Example:

1) double x=15.26;
int y=x; *//Error- because data type of 'x' and 'y' are different.*

To store value of 'x' into 'y', we have to convert data type of 'x' into data type of 'y' as follow,

int y= **(int)** x; *//here, the data type of 'x' is converted into data type of 'y' using **(int) cast operator***

2) int x=65;
char y = **(char)** x;*//here, the data type of 'x' is converted into data type of 'y' using (char) cast operator*

Type casting: (Type conversion)

- Converting one data type into another data type is called "Type casting" or "Type-conversion".
- We can convert the values from one type to another explicitly using the cast operator as follows.
- Syntax for type casting:
  (type_name) expression;
- Here, type_name is any valid data type into which we can convert value of expression.
-

Following program shows type casting that convert char data type into int data type.

```
public class typeCast
 {
   public static void main(String []args )
   {
     char ch= 'B';
     int p;
     p = (int) ch;          //type casting
     System.out.println("ASCII value="+p);
    }
}
```

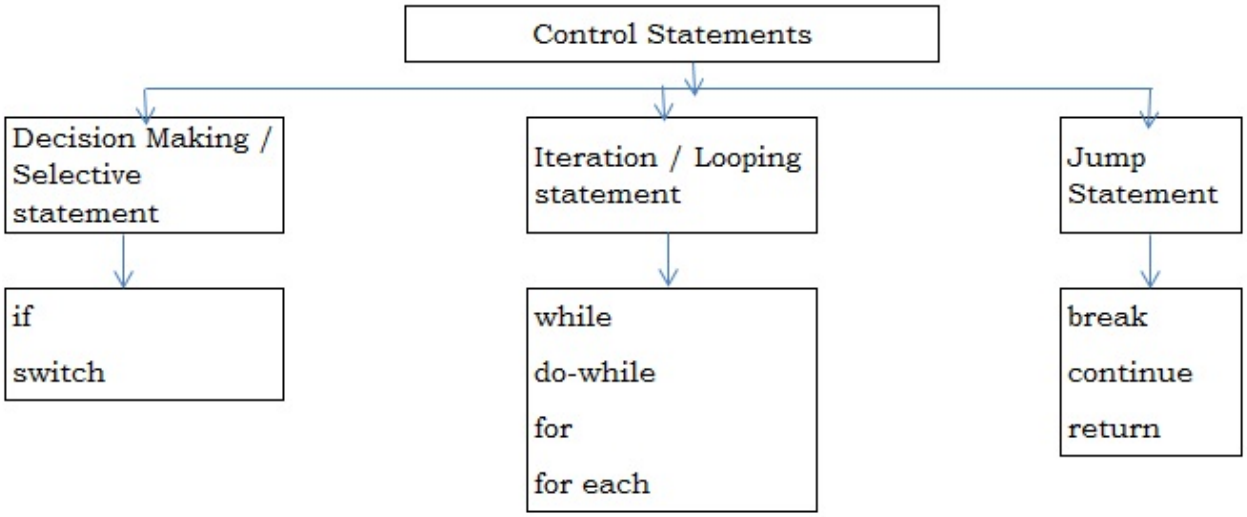
Following program shows type casting that convert int data type into float data type.

```
public class typeCast
 {
   public static void main(String []args )
   {
       int a=5,b=2;
       float p;
       p = (float) a/b;          //type casting
       System.out.println("Division="+p);
    }
}
```

Control Statements in Java:

The statement that controls the flow of execution of program is called as "Control statement" or "Control Structure".

The following tree diagram shows control statements in Java language:



(Note that: All the Control Statement in Java is same as that of C/C++ language therefore refer notes of C/C++ language)

for-each loop:

- This loop is specially designed to handle elements of 'collection'.
- *Collection* represents a group or set of elements or objects.

For example: We can take an 'array' as collection because 'array is set or group of elements'.

- Also, any class in 'java.util' package can be considered as 'collection' because any class in 'java.util' package handles group of objects such as 'stack', 'vector', 'LinkedList' etc.
- The *for-each* loop repeatedly executes a group of statements for each element of the collection.
- The execution of for-each loop depends upon total number of elements or objects present in the collection.

Syntax:

```
for (datatype  var : collection )
{
    Statements;
}
```

Here,

'var' is an identifier which represents each element of collection one by one. Suppose, the collection has 5 elements then this loop will be executed 5 times and 'var' will *store each element of collection one by one*.

'datatype' is any valid data type in Java which is same as collection.

'collection' is any collection such as array, stack, linked list, vector etc.

```
// Program that demonstrate use of for-each loop
class       Demo1
{
     public  static void main(String   [ ] args)
     {
               int     arr[ ]={5,6,7,8,9};

               for (int  i : arr )          //  'i' represents each element of 'arr'
               {
                  System.out.println(i);
               }
     }
}
```

'continue' statement:

- 'continue' statement is specially used in looping statement.
- When 'continue' statement is executed then *control transferred back to check the condition in loop* and *rest of statements are ignored or skipped*.

```
// Syntax or execution of 'continue'  statement

While ( condition1 )
{
          if ( condition2 )
          {
            continue;
          }
              -----------
              -----------
}
```

```
// Program that demonstrate use of 'continue' statement
class      myloop
{
     public  static void main(String   [  ]st)
     {
          int i=1;
                while(i<=10)
                {
                   if(i<=5)
                   {
                      System.out.println(i);
                      i++;
                      continue;
                   }
                   else
                   {
                       i++;
                   }
          }
     }
}
```

OUTPUT: 1 2 3 4 5

Reading Input using '*Scanner*' class of '*java.util*' package:

- We can read varieties of inputs from keyboard or from text file using methods of '*Scanner*' class.
- *Scanner* class belongs to 'java.util' package.
- When *Scanner* class receives input, it breaks the input into several pieces, called 'tokens' and these tokens can be retrieved using object of *Scanner* class.
- Note that: Following methods of *Scanner* class are ***non-static*** therefore they are called or accessed with the help of *object* of *Scanner* class.

We can create object of Scanner class as follows:

```
Scanner obj=new Scanner(System.in);
```

Here, '**obj**' is object of *Scanner* class.
'**System.in**' represents *InputStream*, which is by default represents standard input device i.e. Keyboard.
There are several methods of *Scanner* class used to take different inputs as follows:

Method	Working
next( )	It is used to read single word string
nextLine()	It is used to read single **string having multiple word**.
nextByte( )	It is used to read single byte type value
nextInt( )	It is used to read single integer type value
nextFloat( )	It is used to read single Float type value
nextLong( )	It is used to read single Long type value
nextDouble( )	It is used to read single Double type value
nextShort( )	It is used to read single short type value

Following program demonstrate the use of different methods of *Scanner* class.

```
import java.util.*;
class   cricket
{
        byte    no;
        String  name;
        long    contact;
        int     t_sc;
        short   t_wk;
        float   ball_avg;
        double  bat_avg;
    void get()
    {
       Scanner  sc=new  Scanner(System.in);
          System.out.print("Enter Cricketer No= ");
           no=sc.nextByte();
           System.out.print("Enter Cricketer Name= ");
           name=sc.next();
           System.out.print("Enter Cricketer Contact No= ");
           contact=sc.nextLong();
           System.out.print("Enter Cricketer Total Score= ");
           t_sc=sc.nextInt();
           System.out.print("Enter Cricketer Wickets= ");
           t_wk=sc.nextShort();
           System.out.print("Enter Ball AVG= ");
           ball_avg=sc.nextFloat();
           System.out.print("Enter Batting AVG= ");
           bat_avg=sc.nextDouble();
    }
    void show()
    {
        System.out.println("----------------------------------------");
           System.out.println("CricketerNO="+no);
           System.out.println("CricketerName="+name);
           System.out.println("ContactNo="+contact);
           System.out.println("Total Score="+t_sc);
           System.out.println("Total Wickets="+t_wk);
           System.out.println("Balling AVG="+ball_avg);
           System.out.println("Batting AVG="+bat_avg);
    }
   public static void main(String []args)
   {

        cricket  obj=new  cricket();
        obj.get();
        obj.show();    }
}
```

User defined methods in JAVA:

- Like C or C++ language, JAVA language also has 4 types of methods depending upon parameter acceptance or not and value return or not.
- Following program shows defining 4 types methods in Java language.

```
import     java.util.Scanner;
public    class     FunctionDemo
{
          int     x,y,z;
          Scanner   sc=new Scanner(System.in);
   void      add(int   a,int   b)       //with arg. without return value
   {
          int c;
          c=a+b;
          System.out.println("Addition="+c);
   }
   int     sub(int   a,int   b)      //with arg. with return value
   {
          int c;
          c=a-b;
          return(c);
   }
   int     multi()       //without arg. with return value
   {
          System.out.println("Enter Two Numbers=");
          x=sc.nextInt();
          y=sc.nextInt();
          z=x*y;
          return(z);
   }
   void   division()      //without arg. without return value
   {
          System.out.println("Enter Two Numbers=");
          x=sc.nextInt();
          y=sc.nextInt();
          z=x/y;
          System.out.println("Division="+z);
   }
   public static void main(String []args)
   {
          FunctionDemo    obj=new     FunctionDemo();
          System.out.println("Enter two number");
          obj.x=obj.sc.nextInt();
          obj.y=obj.sc.nextInt();
          obj.add(obj.x,obj.y);
          System.out.println("Enter two number");
          obj.x=obj.sc.nextInt();
          obj.y=obj.sc.nextInt();
          obj.z=obj.sub(obj.x,obj.y);
          System.out.println("Subtraction="+obj.z);
          obj.z=obj.multi();
          System.out.println("Multiplication="+obj.z);
          obj.division();
   }
}
```

Core JAVA Theory Assignment No: 01

1) What is Java? Explain its various features OR properties.
2) What is Java? Write its evolution (History of Java) . And list out its drawbacks.
3) Write difference between C++ and Java.
4) Explain different components of JDK with their use.
5) Explain JVM architecture. **OR** Explain working of JVM **OR** How JVM works?
6) What are the different naming conventions used in Java?
7) Why Java does not support for Pointers?
8) What are Java Tokens? List out its different tokens.
9) What is variable? Explain instance variable, class variable and local variable with example.
10) Explain different operators in Java.
11) Explain 'instanceof' operator in java with one example.
12) Explain while, do-while, for and for-each loop in Java with one example.
13) What is type casting? How type casting is done in Java?

Core JAVA Practical Assignment No: 1

Note: Implement following programs by using command line Argument in JAVA

1) Write a program to print First name, Middle name and Last name of employee.
2) Write a program which prints first 'n' numbers.
3) Write a program which find sum of first 'n' numbers.
4) Write a program which find sum of even numbers and odd numbers from 1 to N.
5) Write a program which prints factors of entered number.
6) Write a program which check entered number is Perfect or not.
7) Write a program which find sum of digits (digit sum) of entered number
8) Write a program which check entered number is Armstrong or not.
9) Write a program which reverses the entered number.
10) Write a program which check entered number is Palindrome or not.
11) Write a program which check entered number is Prime or not.
12) Write a program which finds factorial of an entered number.
13) Write a program which prints Fibonacci series up to 'n' numbers.
14) Write a program to check entered number is Strong or not.

    *(**Hint:** Strong number is a special number whose sum of the factorial of digits is equal to the original number For Example: 145 is strong number. Since, 1! + 4! + 5! = 145)*

15) Write a program to check entered number is Magic or not.

    *(**Hint:** For example, 325 is a magic number because the sum of its digits (3+2+5) is 10, and again sum up the resultant (1+0), we get a single digit (1) as the result. Hence, the number 325 is a magic number. Some other magic numbers are 1234, 226, 10, 1, 37, 46, 55, 73, etc.)*

Core JAVA Practical Assignment: 02

Note: To accept inputs form keyboard use methods of 'Scanner' class of java.util

1) Write a program to find addition, subtraction, multiplication, division of two numbers.
2) Write a program to find average of five numbers.
3) Write a program to find area of circle.
4) Write a program to find circumference (perimeter) of circle.
5) Write a program to find area of triangle.
6) Write a program which accepts six subject marks and calculates total marks and percentage of student.
7) Write a program to swap two integers.
8) Write a program to check entered number is positive or negative.
9) Write a program to check entered number is even or odd.
10) Write a program to find maximum number between three numbers.
11) Write a program to find minimum number between three numbers.
12) Write a program that demonstrate the use of 'instanceof' operator
13) Write a program that demonstrates the use of 'cast' operator.
14) Write a program which calculates total marks and percentage obtained in six subjects and also display grade of student according to following table:

Percentage	Grade
0 to 39.99	Fail
40 to 49.99	Third
50 to 59.99	Second
60 to 69.99	First
70 to 100	Distinction

15) Write a program which calculates income tax corresponding to Following table:

Income	Tax
0 to 150000	0%
150001 to 300000	10%
300001 to 500000	20%
500001 and above	30%

16) Write a program which calculates telephone bill corresponding to following table:

Unit Consumed	Rate/unit in RS.
0 to 200	1.00
201 to 350	1.20
351 to 500	1.50
501 and above	1.75

17) Write a program which take single digit number as input and print corresponding number into word.
18) Write a program menu driven program to find out area of circle, circumference (perimeter) of circle, area of triangle and area of square
19) Write a menu driven program for:
    1: Addition of two numbers.
    2: Subtraction of two numbers.
    3: Multiplication of two numbers.
    4: Division of two numbers.
    5: Modulation of Two numbers.
20) Write a program to print First name, Middle name and Last name of employee.
21) Write a program which find sum of even numbers and odd numbers from 1 to 20.
22) Write a program which prints first 'n' numbers.
23) Write a program which find sum of first 'n' numbers.
24) Write a program which prints factors of entered number.
25) Write a program which check entered number is Perfect or not.

26) Write a program which find sum of digits (digit sum) of entered number
27) Write a program which check entered number is Armstrong or not.
28) Write a program which reverses the entered number.
29) Write a program which check entered number is Palindrome or not.
30) Write a program which check entered number is Prime or not.
31) Write a program which finds factorial of an entered number.
32) Write a program which prints Fibonacci series up to 'n' numbers.
33) Write a program to check entered number is Strong or not.
34) Write a program to check entered number is Magic or not.
35) Write a program to find all Armstrong numbers from 1 to 1000
36) Write a program to find all Prime numbers from 1 to 1000
37) Write a program to find all palindrome numbers from 500 to 700
38) Write a program which prints multiplication table up to 10.

Core JAVA Practical Assignment: 03

Que. Write the program that prints following pattern:

1)

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

2)

```
5 4 3 2 1
5 4 3 2
5 4 3
5 4
5
```

3)

```
A B C D C B A
A B C   C B A
A B       B A
A           A
```

4)

```
        A
      A B
    A B C
  A B C D
A B C D E
```

5)

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
1 2
1 2 3
1 2 3 4
1 2 3 4
1 2 3 4 5
```

6)

```
1
2  3
4  5  6
7  8  9  10
```

7)

```
    1
  0   1
0   1   0
1   0   1   0
```

8)

```
@
@@
@@@
@@@@
@@@@@
@@@@
@@@
@@
@
```

9)

```
@
@@
@@@
@@@@
@@@@@
@@@@@@
```

10)

```
@
@@
@@@
@@@@
@@@@@
@@@@@@
@@@@@
@@@@
@@@
@@
@
```

11)

```
$ $ $ $ $ $ $ $ $ $
$ $ $ $ $ $ $ $ $
$ $ $ $ $ $ $ $
$ $ $ $ $ $ $ $
$ $ $ $ $ $ $
$ $ $ $ $ $
$ $ $ $
$ $ $
$ $
$
```

Arrays in JAVA:

INTRODUCTION:

- We know that, the concept of variable is introduced **to store the data**.
- But single variable can store only one value at a time, this is the drawback of variable. To overcome the drawback of single variable the concept of *Array* is introduced.
- That is, *array* is also single variable but it has an ability to store multiple (more than one) value at a time.

Definition:

- "An Array is collection of elements or items having **same data type** referred by common name"
  - **OR**
- "An Array is collection of homogeneous (having same type) elements or items referred by common name"
- In an array the individual element is accessed with the help of integer value and is called **subscript or index.** Also, all elements of array are stored in **continuous memory** allocation.
- *Note that:*
  - *We know that, In C/C++ language, memory for array is get allocated at compile time (i.e. static memory allocation)*
  - *But, in JAVA everything is dynamic i.e. for variable, array, objects etc. memory is get allocated at run time (Dynamic memory allocation) by JVM*

Types of Array:

Depending on number of subscripts used in array, array having three types:

1) One Dimensional array: (1 D array)

The array having **only one subscript** is called as "*One dimensional array*"

Declaration Syntax:

```
datatype    array_name[ ]=new datatype [Size];
                    OR
datatype    [ ]array_name=new datatype [Size];
```

Here;

datatype is any valid *data type* in JAVA language

array_name is name of array which is an identifier.

'*size*' is integer value that denotes total number of elements stored in array

'*new*' is an operator.

e.g.

1) int x[ ]=new int[5];*//declares array 'x' & allocates memory for 5 integers*

   OR

   int [ ]x =new int[5];

here ;

'x' is array which can holds(stores) **five** integers at a time.

2) int marks[ ]; *//declares array 'marks'*

   marks=new int[5]; *//allocates memory for 5 integers*

Initialization of One dimensional array:

One dimensional array can be initialized as fallow;

e.g. int p[ ] = { 10 , 20 , 30 , 40 , 50 } ;

Here;

'p' is integer type array which stores five integers at a time. The storage of all elements in array 'p' is shown in following figure:

Index →	*0*	*1*	*2*	*3*	*4*
Elements of 'P'	**10**	**20**	**30**	**40**	**50**
Address →	110	114	118	122	126

In above figure the elements 10, 20, 30, 40, 50 are stored in array 'p' at individual index. That is the element 10 is stored at index 0, 20 is stored at index 1 like that, 50 is stored at index 4. Also all elements are stored in continuous memory location.

Note:

Index is integer value which is nothing but position of the element in an array.

2) Two Dimensional array: (2D array)

The array having **two subscripts** is called as "Two dimensional array or matrix"

The two dimensional array is used to perform matrix operations.

Declaration Syntax:

```
datatype      array_name[ ][ ]=new  datatype [Size1][Size2];
                          OR
datatype      [ ][ ]array_name=new  datatype [Size1][Size2];
```

Here;

datatype is any valid *data type* in Java language

array_name is name of array which is an identifier.

'*Size1'* is integer value that denotes total number of rows in array

'*Size2'* is integer value that denotes total number of columns in array

E.g.

1) `int x[ ][ ]=new int[3][2];`

OR

`int [ ][ ] x=new int[3][2];`

here ;

'*x'* is array which can holds total 6 integers at a time.

3 is *rowsize* i.e. there are 3 rows in matrix 'x'

2 is *columnsize* i.e. there are 2 columns in matrix 'x'

Initialization of Two dimensional array:

Two dimensional array can be initialized as follow;

e.g. 1) `int p[ ][ ]={{3,4},{2,9},{7,6}};`

2)
```
int z[ ][ ] ={  {  5, 3 , 6 } ,
                {  1, 7 , 8 } ,
                {  9, 4 , 2 }  };
```

Here;

'z' is integer type two dimensional array which stores nine integers at a time. The storage of all elements in array 'z' is shown in following figure:

row index ↓	**0**	**1**	**2** ← column index
0	5	3	6
1	1	7	8
2	9	4	2

In above figure the individual element of array 'z' is also accessed by following way:

The element 8 can be accessed as z[1][2]

The element 7 can be accessed as z[1][1]

The element 9 can be accessed as z[2][0] etc.

like that we can access all individual elements of matrix 'z'

3) Multi-Dimensional array:

"The array having **more than two subscripts** is called as *multi-dimensional* array"

Declaration Syntax:

```
datatype array_name[ ][ ]……[ ]=new datatype [Size1][Size2]……[SizeN];
                              OR
datatype [ ][ ]……[ ]array_name=new datatype [Size1][Size2]……[SizeN];
```

here;

datatype is any valid *datatype* in Java language.

array_name is an identifier (name given by programmer)

size1, size2,……sizeN are **integer constants** and that denotes total number of elements stored in an array.

e.g. 1) int z[][][]=new int[2]**[4][3]**;

Above array is the multidimensional array having three subscripts and which stores 12 integer values at a time.

*That is above multi-dimensional array stores **2 sets of 4*3 matrices**.*

2) float x[ ][][][]=new float[2][3][2][2];

The above array 'x' is also multidimensional array having four subscripts and which stores 24 float values at a time.

Initialization of Multi-dimensional array:

Multi-dimensional array can be initialized as follows:

Example:

1) int x[][][]={{{1,2},{4,5},{7,8},{3,6}},
{{11,12},{14,15},{17,18},{13,16}} };

In above multi-dimensional array, elements are stored in following manner;

X[0] ⇨

	0	*1*
0	**1**	**2**
1	**4**	**5**
2	**7**	**8**
3	**3**	**6**

x[1] ⇨

	0	*1*
0	**11**	**12**
1	**14**	**15**
2	**17**	**18**
3	**13**	**16**

From above, multi-dimensional array we can access individual element as follow:

X[0][1][0] access 4
X[0][2][1] access 8
X[0][3][0] access 3
X[1][1][1] access 15
X[1][2][0] access 17
X[1][3][1] access 16

'length' property of array:

- 'length' is property of an array *returns one integer value which is nothing but **size of array**.(in case of 1D array)*
- We use this property as follows:

int var = arrayname.length;

Here, 'var' is an *int* type variable which stores size of array return by *arrayname.length* poperty.
Exampl:

1) int arr[ ]=new int[10]; *//declares array with 10 size*
   arr[0]=45;
   arr[1]=90;
   int sz=arr.length; *//returns array size*
   System.out.print("Array Size="+sz);

OUTPUT:

Array Size=10

Note that:

- In above example 'arr' is array declared with 10 size. And arr[0] and arr[1] are initialized with values 45 and 90 respectively. But '*arr.length*' statement returns value 10 which is size of array (Since, *'length' property returns **size of array (first subscript value)**. It **not** returns total number of array elements*)
- *Also, in case of 2D or Multi-dimensional array, 'length' property returns **number of rows of the array (first subscript value)**.*

Following program shows use of 'length' property of array.

```
public class Myarr
{
   int arr[]=new int[5];
   int arr1[][]=new int[2][3];
   int arr2[][][]=new int[4][5][6];
   void get()
   {
    int i=arr.length;
    int j=arr1.length;
    int k=arr2.length;
    System.out.println("Size of 1D="+i);
    System.out.println("Rows of 2D="+j);
    System.out.println("Rows of MD="+k);
   }
  public static void main(String []ar)
  {
    Myarr p=new Myarr();
    p.get();
  }

}
OUT PUT:
            Size of 1D=5
            Rows of 2D=2
            Rows of MD=4
```

Core JAVA Practical Assignment: 04

Array in JAVA

1) Write a program to accept 10 elements of array and find maximum and minimum in it.
2) Write a program to find sum of all array elements.
3) Write a program to accept 10 elements and find sum of all even numbers and average of all odd numbers in it.
4) Write a program to find addition two matrices.
5) Write a program to find subtraction two matrices.
6) Write a program to find Multiplication two matrices.
7) Write a program to find sum of only even numbers of matrix having order 3X4.
8) Write a program to find maximum and minimum number in matrix.
9) Write a program which accepts any square matrix as input and find sum of only diagonal elements.

What is Object Oriented Programming (OOP)?

- Object oriented programming (OOP) is a programming paradigm (Model) that depends on the concept of **classes and objects**.
- It is used to develop a software program which is simple, reusable using classes, which are used to create individual instances i.e. objects.
- There are many object-oriented programming languages like JavaScript, C++, C#, Java, PHP, Python etc.

Principles of OOP (OOP's Concepts):

- OOP totally depends on several OOP principles like class, object, data encapsulation, data abstraction, polymorphism, inheritance, message passing, Persistence etc.
- As per **our syllabus**, Let's see only **Class and Object** principles in details-

1) Class:

- 'Class' is user defined data type which is collection of variables (instance variables) and methods.
- That is, these variables are called 'properties or attributes and methods are called 'actions'
- Also, 'Class' is blue print for object because everything in class can also hold by object. i.e. class decides how the object will be?
- Since, all variables and methods are also available in objects, because they are created from class therefore, they are called 'instance variable' and 'instance method'.

Consider, following defined class for 'person'.

```
class          person
{
      int      age;
      String   name;      //Properties- i.e. instance variables of class

      void     talk( )    // action- i.e. instance method
      {
            System.out.println("Hi, My name is "+name);
            System.out.println("My age is "+age);
      }
}
```

In above, example *person* is class that consists of instance variables (properties) *age, name* and having instance method (action) *talk( )*.

2) Object:

- 'Object' is basic run time entity that holds entire data (variables and methods) of class.
- Object is also called as 'instance' of class.
- When object is created then that created object is stored in 'Heap area' by JVM.
- Also, whenever an object is created then JVM allocates separate memory reference (address) for created object called 'hash code'
- This 'hash code' is unique hexadecimal number created by JVM for every object.
- We can also see *hash code* of every created object using *hashCode()* method of *Object* class of *java.lang* package.

Syntax to create object:

```
class_name      obj= new     class_name( );
```

Here, *class_name* is name of the class which is an identifier
obj is created object of class.

Purpose to create object:

There are following purpose to create the object:

1) To store entire data (variables and methods) of class.
2) To access variable or to make call for methods of class from outside class.

Following program demonstrate the use of *hashCode()* method.

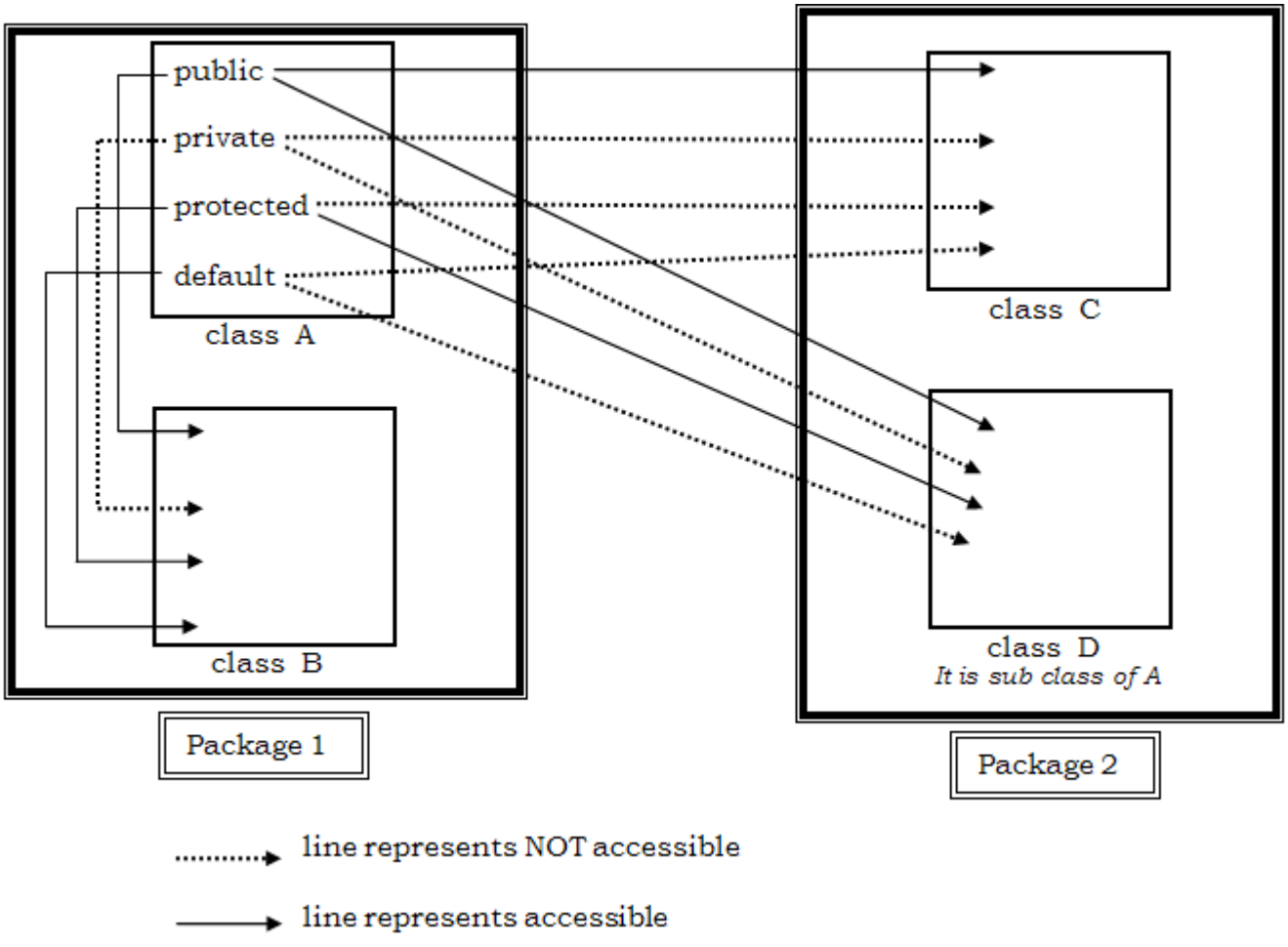
```
Class        Demo
{
     public    static   void   main(String [ ] args)
     {
               Demo   a=new   Demo();
               Demo   b=new   Demo();
               System.out.println("First Object's Hash Code="+a.hashCode( ));
               System.out.println("Second Object's Hash Code="+b.hashCode( ));
     }
}

OUT PUT:
     First Object's Hash Code=53ab83406
     Second Object's Hash Code=1b6164678
```

Access Specifier (Member Access Controls) in Java:

- In **Java**, an **access specifier** (also called an **access modifier**) defines **where a class, method, variable, or constructor etc. can be accessed from**.
- Access specifier controls the **visibility** and **access level** of class members.
- Use of access modifiers are-
  - Provide security – prevent unauthorized access to data
  - Support encapsulation – hide internal implementation details
  - Control code accessibility – define clear boundaries between classes and packages
  - Improve maintainability- reduce accidental misuse of code
- There are 4 access specifiers in Java language VIZ.

  1) public 2) private 3) protected 4) default

  Following figure shows different access Specifier with their accessibility:



Let us see all these access Specifier in details:

1) public:

- 'public' members of class are accessible everywhere therefore they have **global scope**.
- They are accessible:
  1) Within methods of same class.
  2) Within methods of sub classes.
  3) Within methods of other class of same package.
  4) Within methods of class of another package.

2) private:

- 'private' members of class are only accessible by methods of same class therefore they have class scope. That is private members are not accessible outside the class.
- They are accessible:
  1) Within methods of same class.
- They are NOT accessible:
  1) Within methods of sub classes.
  2) Within methods of other class of same package.
  3) Within methods of class of another package.

3) protected:

- The accessibility of 'protected' members of class is given as follow;
- They are accessible:
  1) Within methods of same class.
  2) Within methods of sub classes.
  3) Within methods of other class of same package.
  4) Within methods of classes of other package (Since, other package class should be sub class)
- They are NOT accessible:
  1) Within methods of classes of other package (If other package class should **NOT** be sub-Class)

4) default:

- If no access specifier is written by the programmer, then Java compiler uses 'default' access specifier.
- The 'default' members of class should be accessible by the methods of class of same package i.e. they are not accessible out the package. Therefore 'default' access specifier has 'package' scope.
- They are accessible:
  1) Within methods of same class.
  2) Within methods of sub classes.
  3) Within methods of other class of same package.
- They are NOT accessible:
  1) Within methods of classes of another package.

Methods in Java:

In Java, there are two kinds of methods available in class viz-

1) Instance Methods (Non-Static Methods) 2) Class Methods (Static Methods)

Let us see all these methods in details:

1) Instance Methods (Non-Static Methods):

- The method which is defined within class body ***without*** using keyword 'static' are called as 'Instance methods or Non-static methods'.
- These types of methods are act on 'instance variable' (object) of class.
- They are called 'Instance' method because their content is stored in instance (object) of class.
- Instance methods are called with the help of object of class using syntax:

  object.method(arg1,arg2, ----);
- One specialty of instance method is that they are capable to access instance variable (non-static variable) and class variable (static variable) directly.

Types of Instance Methods:

Further, Instance methods are of two types:

1) Accessor Method 2) Mutator Method

Let's see these methods in details:

1) Accessor Method: (Getter Method)

- This type of instance methods is *only* **access or read** value of instance variables i.e. they **do not modify** value of instance variable are called as "Accessor Method".

- This method is also called the **Getter method**. Because, this method returns the variable value.

2) Mutator Method: (Setter Method)

- This type of instance methods is **access and modify** value of instance variable are called as "Mutator Method".
- This method is also called the Setter method. Because, this method sets the variable value.

Following program shows the use of Accessor and Mutator methods.

```
class    Access
{
     int       age;
     String   name;
     int      getAge( )   //Accessor method
     {
              return(age);
     }
     String   getName()    //Accessor method
     {
              return(name);
     }
     void    setAge(int      x)    //mutator method
     {
                 age=x;
      }
     void   setName(String    nm)//mutator method
     {
                 name=nm;
     }
     public  static  void  main(String  [ ] args)
     {
         Access   t=new   Access();
         t.setAge(55);
         t.setName("RAJESH");
         System.out.println("Age="+t.getAge());
         System.out.println("Name="+t.getName());
         }
     }
```

2) Class Methods (Static Methods):

- The method which is defined within class body using keyword 'static' is called as 'class methods or static methods.'
- These types of methods are act on 'class variable (static variable)' of class.
- They are called 'Class' methods because they are defined using 'static' keyword & it is related with class.
- Class methods are called with the help of class name using syntax:

ClassName.method(arg1,arg2, ----);

- Class methods are capable to access only class variable (static variable) directly.
- Still, if we want to access instance variable inside class method then it can be accessed only by using object of class.

Following program shows use of class method:

```
public   class      Empolyee
{
     static  int       no;
     static  String   name;
     static  void  get()
     {
        no=65;
        name="KIRAN";
     }
     static   void   show()
     {
          System.out.println("Number="+no);
          System.out.println("Name="+name);
     }
    public   static   void   main(String  []args)
     {
              Empolyee.get( );          //class methods accessed with class name.
              Empolyee.show( );
     }
}
```

(**HOME WORK**: *Write Difference between Instance Methods and Class Methods)*

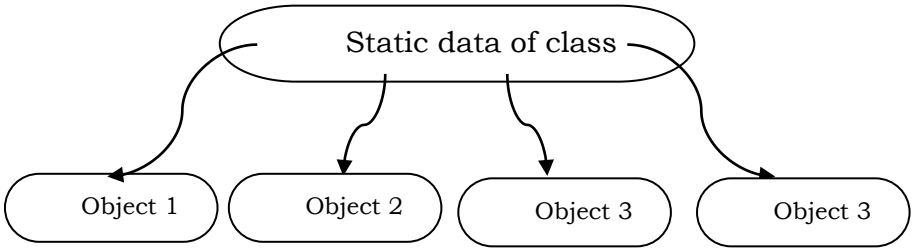
Static keyword:

- We know that, for single class we can create multiple objects. And each object holds their individual record. But there are some data which are common to all objects in such situation that data can be made as static using 'static keyword'.
- Once the class data is made as static then it will become common to all objects of class and it is shared by all objects of class.
- Only one copy of static data is maintained for entire class therefore it is shared among all objects.
- Static data are initialized at once therefore it is **used like a counter.**
- Static data are stored separately rather than as a part of an object.
- Syntax to declare static variable:

```
static      datatype    variable;
```

Here, 'static' is keyword which is used to declare member as static
'datatype' is any valid data type in Java.
'variable' is name of static variable.

Following fig. shows sharing of static data by different objects of class:



In above fig. static data of class is common to all objects Object1, Object2, Object3, Object4. Or the same copy of static data is shared among all objects.

*Observe OUTPUT of following program **without** **static** keyword:*

```
import   java.util.Scanner;
class   statDemo
{
   String   name;
   void get()
   {
      Scanner sc=new Scanner(System.in);
      System.out.println("Enter name=");
      name=sc.next();
   }
   void show()
   {
      System.out.println("Name="+name);
   }
   public static void main(String ar[])
   {
      statDemo   a=new statDemo();
      statDemo   b=new statDemo();
      a.get();
      b.show();
   }
}
```

OUTPUT:

```
Enter name= JIBRAIL
Name=null
```

*Observe OUTPUT of following program **with** **static** keyword:*

```
import   java.util.Scanner;
class   statDemo
{
  static String name;
   void get()
   {
      Scanner sc=new Scanner(System.in);
      System.out.println("Enter name=");
      name=sc.next();
   }
   void show()
   {
      System.out.println("Name="+name);
   }
   public static void main(String ar[])
   {
      statDemo   a=new statDemo();
      statDemo   b=new statDemo();
      a.get();
      b.show();
   }
}
```

OUTPUT:

```
Enter name= JIBRAIL
Name=JIBRAIL
```

Following program demonstrate the use of static variable that counts total object created for class:

```
class count
{
    static int cnt;
    count( )
    {
       cnt++;
    }
    static void show( )
    {
          System.out.println("Total Objects= "+cnt);
     }
    public static void main(String [ ]args)
    {
          count a=new count( );
          count b=new count( );
          count c=new count( );
          count d=new count( );
          count.show( );
    }
}
```

OUTPUT: Total Objects= 4

Method Overloading:

- Method overloading is type of compile time polymorphism where we can define many methods having same name but all these methods are differed from each other according to their signature (type of argument and number of arguments accept by method).
- It is type of compile time polymorphism; hence all overloaded methods are get selected by JVM at compile time of java program.
- **Note that:** The functionality of overloaded method should be same.
- Following program shows method overloading that finds addition of two numbers:

```
public class MethodOverload
{
   int a,b,c;
   void add( )
   {
    a=5;
    b=7;
    c=a+b;
    System.out.println("Addition="+c);
   }
   void add(int x)
   {
    a=10;
    c=a+x;
    System.out.println("Addition="+c);
   }
   void add(int x,int y)
   {
    c=x+y;
    System.out.println("Addition="+c);
   }
   public static void main(String []args)
   {
    MethodOverload p=new MethodOverload();
    p.add();
    p.add(15);
    p.add(45,65);
   }
}
```

Constructor:

- The constructor is special method whose name is same as that of class name.
- The main task of constructor is to initialize values for object of its class.
- It is called constructor because it constructs the values for object of the class.
- The constructors are implicitly invoked by the compiler whenever an object of class is created.
- Example:
  Consider, there is one class having name "student" then its constructor is given as:

```
class student
{
    student( )    // constructor
    {
        -------
        -------
    }
}
```

Properties/Characteristics/Features of Constructor:

1) Constructor name is same as that of class name.
2) Constructor does not return any value but it can accept some parameter as input.
3) Constructor does not have any return type even void.
4) Constructors are implicitly invoked by the compiler whenever object of class is created.
5) Constructor can be overloaded.
6) Constructors are not static.

Types of constructor:

Depending upon arguments passed to constructor, it has two types.

1) Default Constructor
2) Parameterized constructor

***NOTE**:*

1) *Java language does not support for copy constructor.*
2) *If we specify return type for constructor then Java compiler treats that method as normal method.*

1) Default constructor:

- The constructor that does not have any argument or parameter as input, such constructor is called as "Default Constructor".
- Default constructor is implicitly invoked by compiler whenever object of class is crated

Syntax:

```
class  class_name
{
    class_name( )      // default constructor.
    {
        -------
        -------
    }
}
```

Following program shows implementation of default constructor:

```
class  student
{
  int  roll;
  String  name;
  double  per;
   person( )   // default constructor
  {
    roll=70;
    name= "Sachin";
    per= 68.50;
  }
  void  show( )
  {
    System.out.println("Roll="+age);
    System.out.println("Name="+name);
    System.out.println("Percentage="+per);
  }
  public  static  void  main(String [ ] args )
  {
      student  x=new  student( );  //call to def. constructor
      x.show( );
  }
}
```

2) Parameterized constructor:

- The constructor that accepts any argument or parameter as input, such constructor is called as "parameterized Constructor".
- Parameterized constructor is implicitly invoked by compiler when we pass some values direct to object at the time of its creation.

Syntax:

```
class  class_name
{
    class_name( int  x, String  y)      // parameterized constructor.
    {
        -------
        -------
    }
}
```

Following program shows implementation of parameterized constructor:

```
class  person
{
  int  age;
  String  name,add;
  double  sal;
  person(int  a, String  n ) //para.constructor
  {
    age=a;
    name=n;
    add= "Pune";
  }
  void  show( )
  {
    System.out.println("Age="+age);
    System.out.println("Name="+name);
    System.out.println("Address="+add);
    System.out.println("Salary="+sal);
  }
  public  static  void  main(String [ ] args )
  {
     person  x=new person(45, "Raj" );  //call to parameter constructor
```

```
        sal= 4568.50;
      }
```

```
      x.show( );
    }
   }
```

Overloading of Constructor (Multiple Constructor in Class):

- We know that in case of Method overloading, we can define or implement many methods with same name within same class, but all these methods are differed from each other according to their signature (Type of argument and number of arguments accept by methods).
- Like method overloading, we are also able to overload the constructor.
- Constructor overloading means implementing or defining multiple constructors for single class. But all these constructors are differing from each other according to their signatures.
- Following program shows constructor overloading:

```
class          person
{
   int       age;
   String   name,add;
   double  sal;

    person( )    // default constructor
    {
      age=70;
      name= "Sachin";
      add= "Pune";
      sal= 4568.50;
    }
  person(int  a, String  n ) //para.constructor
  {
      age=a;
      name=n;
      add= "MUMBAI";
      sal= 54508.75;
  }
```

```
  void  show( )
  {
    System.out.println("Age="+age);
    System.out.println("Name="+name);
    System.out.println("Address="+add);
    System.out.println("Salary="+sal);
  }
 public   static  void    main(String [ ]  args )
 {
    person   p=new  person( );
    person   q=new person(45, "Raj" );
    p.show( );
    q.show( );
  }
 }
```

What is the destructor in Java?

- We know that, destructor is a method that destroy (release the memory) the object. It is opposite of constructor.
- In JAVA there is **no destructor like in C++** but it has a special method called 'finalize()' that automatically gets called when an object is no longer used (i.e. destroyed)
- When an object completes its life-cycle the garbage collector automatically deletes that object and de-allocates or releases the memory occupied by the object.
- In Java, automatically memory management is done by garbage collector and hence Java language does not have concept of destructor. But Java language has finalize( ) method whose work is same as that of destructor
- For automatically memory management, JVM implicitly called gc( ) method. *(we called it explicitly as System.gc())*

finalize( ) Method:

- It is difficult for the programmer to forcefully execute the garbage collector to destroy the object. But Java provides an alternative way to forcefully execute the garbage collector to destroy the object.
- The Java *Object* class provides the *finalize()* method that works the same as the destructor.
- It is a **protected method** of the **Object class** which is defined in the *java.lang* package.
- It can be called only once.
- The syntax of the finalize() method is as follows:

  Syntax:

```
protected void finalize ( )
{
        //resources to be close
}
```

Following program shows use of finalize( ) method:

```
public class DestructorDemo
{
      DestructorDemo()
     {
          System.out.println("This is Constructor");
      }
    protected void finalize()
    {
      System.out.println("Object is destroyed ...");
    }
public class main(String[] args)
{
    DestructorDemo de = new DestructorDemo();
    de.finalize();
}
}
```

'this' keyword in Java:

- In Java, '**this**' is a reference variable that refers to the current object (the object whose method or constructor is being executed) of class.
- '**this**' is not static i.e. it **cannot be used in static methods** of class.
- Commonly '**this**' is used to avoid naming conflicts between formal parameters and instance variables.
- While using 'this' in constructor, it should be the first statement.

Why 'this' keyword is used?

→ Following are the several uses of 'this' keyword:

1) Differentiate instance variables from parameters:

```
class Student
 {
   int roll;
   Student(int roll)
   {
      this.roll = roll;        // this.roll → instance variable, roll → formal parameter
   }
}
```

2) Call another constructor in the same class:

```
class Student
 {
   int roll;
   String name;
    Student()
   {
      this(105, "SANJAY");        // calls parameterized constructor
   }
   Student(int roll ,String name)
  {
      this.id = id;
      this.name = name;
   }
}
```

3) Passing current object as argument:

```
class Test {
   void display(Test obj) {
      System.out.println("Method called");
   }
   void call() {
      display(this);   // passing current object
      }
}
```

4) Return the current object:

```
class Box
 {
   int length;
   Box   setLength(int   length)
    {
      this.length = length;
      return   this;      //returning current object
   }
}
```

Core JAVA Practical Assignment: 05

1) Write a java program to Create a class Student with private variables id, name, and total marks.
Use **getter and setter** methods to:
Assign all values and Display student details in the main method
2) Write a java program to Design a BankAccount class with: Private variable balance, Getter method to return balance and Setter method to deposit money.
3) Write a Java program to demonstrate:Static variable collegeName,Non-static variables studentName and rollNo. Show that static variable is shared among all objects.
4) Write a java program to Create a class Counter that: Uses a static variable to count number of objects created for class and one static method that displays count after creating multiple objects.
5) Write a Java program to overload a method multiply() that: multiply two integers, multiply three integers, multiply two double values.
6) Write a java program to create a Book class with: Default constructor, Parameterized constructor to initialize book name, author and price. And Display() method that display book details.
7) Write a Java program to demonstrate constructor overloading using: constructor with no parameters, constructor with one parameter, constructor with two parameters.
8) Write a java program to Create a class **Product** with **overloaded constructors** to initialize:
   Only productID
   ProductID and ProductName
   ProductID, ProductName and ProductPrice

9) Write a java program where constructor overloading is used to calculate:
   - Area of square
   - Area of rectangle
   - Area of circle

10) Write a Java program that demonstrate the differentiate between instance variable and local variable using *this* keyword
11) Write a Java program demonstrating *this* keyword to pass current object as a method parameter.
12) Write a Java program that demonstrate *this* keyword to Call one constructor from another constructor.

Theory Assignment: 01

2 MARK QUESTIONS	
1) What is Java?	16) What is length property of array?
2) Who developed Java and when?	17) What is an access specifier?
3) What is bytecode?	18) Name different access specifiers in Java.
4) What is JVM?	19) What is a package?
5) What is JDK?	20) What is Scanner class?
6) What is JRE?	21) What is type casting?
7) What is Java API?	22) What is instanceof operator?
8) What is garbage collection?	23) What is new operator?
9) What is portability?	24) What is a static variable?
10) What is platform independence?	25) What is a local variable?
11) What is a class?	26) What is an instance variable?
12) What is an object?	27) What is a keyword?
13) What is a constructor?	28) What is an identifier?
14) What is the purpose of constructor?	29) What is a literal?
15) Why to use this keyword?	30) What is command line argument?

3 MARK QUESTIONS

1) Explain features of Java.
2) Write a short note on history of Java.
3) Explain JVM in brief.
4) Differentiate between JDK and JRE.
5) Explain Java tokens.
6) Explain types of variables in Java.
7) Explain this keyword with example.
8) Explain constructor with example.
9) Explain default and parameterized constructor.
10) Explain access specifiers in details.
11) Explain array in Java with its types.
12) Explain Scanner class methods.
13) Explain instanceof operator with example.
14) Explain type casting with example.
15) Explain new operator.
16) Explain for-each loop with example.
17) Explain continue statement.
18) Explain structure of Java program.
19) Explain difference between #include and import.
20) Explain class and object.

6 MARK QUESTIONS

1) What is Java? Explain its features and advantages.
2) Explain history (evolution) of Java in detail.
3) Explain limitations (disadvantages) of Java.
4) Write a detailed comparison between C++ and Java.
5) Explain Java Development Kit (JDK) and its components.
6) Explain JVM architecture with neat diagram. OR Explain working of JVM step by step.
7) Explain Java Runtime Environment (JRE).
8) Explain structure of Java program with example.
9) Explain Java data types with diagram.
10) Explain Java tokens in detail.
11) Explain operators in Java.
12) Explain bitwise operators with examples.
13) Explain type casting in Java with programs.
14) Explain control statement: if, switch in Java.
15) Explain loops in Java (for, while, do-while, for-each).
16) Explain Scanner class and input handling.
17) Explain arrays in Java (1D, 2D, multi-dimensional).
18) Explain class and object with program.
19) Explain constructors in Java with types.
20) Explain *this* keyword with suitable example.
21) Explain access specifiers in Java with table.
22) Why Java is secure, portable and robust?

MCQ with Answers

1. Java was developed by:
   - Dennis Ritchie
   - **James Gosling**
   - Bjarne Stroustrup
   - Guido van Rossum
2. Original name of Java was:
   - Java
   - **Oak**
   - Green
   - HotJava
3. Java is:
   - Procedure oriented
   - Partially OOP
   - **Pure OOP**
   - Assembly based
4. Bytecode is generated by:
   - JVM
   - JRE
   - JDK
   - **Java Compiler**
5. Which file contains bytecode?
   - .java
   - **.class**
   - .exe
   - .obj
6. JVM stands for:
   - **Java Virtual Machine**
   - Java Variable Method
   - Java Visual Model
   - Joint Virtual Memory
16. Which loop is specially used for collections?
   - for
   - while
   - do-while
   - **for-each**
17. Which package contains Scanner class?
   - java.io
   - java.lang
   - **java.util**
   - java.sql
18. Which method reads integer input using Scanner?
   - next()
   - nextLine()
   - **nextInt()**
   - readInt()
19. Which operator checks object type?
   - new
   - **instanceof**
   - cast
   - typeof
20. Which data type stores true/false?
   - int
   - **boolean**
   - char
   - byte
21. Size of int data type is:
   - 16-bit
   - **32-bit**
   - 64-bit
   - 8-bit

7. Which component makes Java platform independent?
- JDK
- JRE
- JVM
- API

8. Which memory stores objects?
- Stack
- Method Area
- Heap
- PC Register

9. Which keyword is used to create object?
- class
- object
- new
- this

10. Which keyword refers to current object?
- super
- static
- this
- new

11. Constructor name must be same as:
- Method
- **Class**
- Variable
- Package

12. Constructor does not have:
- Name
- **Return type**
- Parameters
- Access specifier

13. Which access specifier has highest visibility?
- private
- default
- protected
- public

14. Which access specifier is package level?
- public
- private
- protected
- default

15. Which variable is shared among all objects?
- Local
- Instance
- Static
- Final

22. Which data type stores single character?
- String
- char
- byte
- boolean

23. Which property gives array size?
- size()
- length()
- length
- count

24. Array index starts from?
- -1
- 0
- 1
- Depends

25. Which statement *skips* remaining statements of loop
- break
- continue
- return
- exit

26. Which Java feature handles memory automatically?
- Pointers
- Destructor
- Garbage Collector
- Malloc

27. Java does not support:
- Inheritance
- Polymorphism
- Pointers
- Encapsulation

28. Which operator converts one data type into another?
- new
- instanceof
- cast
- sizeof

29. Which edition is used for enterprise applications?
- J2SE
- J2EE
- J2ME
- JDK